

CAPSTONE PLAN

LLM Self-Modeled Robot

A physical robot that reads its own telemetry, reasons about the difference between what it predicted and what actually happened, and redesigns itself one generation at a time, with an LLM acting as the designer.

TEAM

David Taylor

Jake Kinchen

Grace Huang

Erick Andrade

The Problem

Every robot carries an internal model of itself: what parts it has, what it can reach, how much force it can apply, and how long a task will take. That model is almost always wrong in small ways. The real friction runs higher than the spec sheet, the assembled arm comes out a few millimeters shorter than the CAD, and the wheels slip on a smooth floor.

Engineers usually fix this by hand. They tune parameters, re-run experiments, and update the firmware. The robot itself has no way to revise its own model, so it keeps failing, quietly or visibly, until a person steps in.

Academic work has studied numerical self-models (Lipson, 2006 to 2019), where a robot infers its own geometry from sensory data. Those models work, but they come out as opaque tensors that a human cannot read and cannot edit without retraining. More recent work (RoboMorph, 2024) uses LLMs to generate robot designs, though only in simulation, with no physical hardware and no telemetry to correct the design against reality.

The open question sits at one specific intersection: no published system combines a language-authored self-model that humans can read and edit, adversarial multi-LLM critique before physical assembly, and per-actuator telemetry feedback that closes the loop on reality. That intersection is still empty.

Our Solution

We build a generational self-model loop that runs on real hardware:

1. An LLM authors a structured self-description of the robot, covering what it is, what it can do, and what it predicts will happen on a task, using a finite typed parts vocabulary (the VEX V5 Classroom Starter Kit).
2. A panel of adversarial critic agents attacks the design before any physical assembly happens.
3. The robot executes real tasks. Per-actuator motor telemetry and camera-based pose data get recorded.
4. The difference between predicted and observed goes back to the LLM, which revises its own self-model.
5. The revised model shapes the next generation's design. A human builds it, and the loop repeats.

The self-model is a versioned JSON document with a `reasoning` field where the LLM explains why it made each structural choice and what evidence led to each revision. Every generation stays human-inspectable.

What We Will Show (June 29 Demo)

By the demo we expect to have completed at least two self-model iterations, carrying the robot from Gen 0 through Gen 2. We present the recorded history of Gen 0 and Gen 1, then run Gen 2 live on the demo surface.

Recorded history of Gen 0 and Gen 1

- Video of the Gen 0 Clawbot executing grab, pull, and throw, with the gap JSON shown beside each run (for example, predicted grip force 14.7 N against observed 11.3 N, an error of -3.4 N).
- The LLM's plain-language read of the Gen 0 residuals and the revision it proposed for the capability layer.
- Video of Gen 1, the first novel configuration, running the same task suite after a human built it from the auto-generated BOM.
- Side-by-side telemetry that compares Gen 0 and Gen 1 gap residuals and marks where the predictions tightened.
- The self-model diff and `reasoning` field for each generation, which together form the audit trail.

Gen 2 live on the demo surface

- Gen 2 is the second self-model iteration, designed from Gen 1's telemetry and checked by the critic panel before assembly.
- The robot runs grab, pull, and throw live, with the predicted values shown on screen before each task.
- Observed telemetry streams in as each task runs, and the live gap appears next to the prediction.
- The audience sees whether Gen 2's self-model called each task correctly, which is the payoff of two rounds of reality correction.
- If time allows, we load the Gen 2 session JSONL into Claude Code live and let it propose the Gen 3 revision on the spot.

Why It Is Ambitious

Everything runs on real hardware. Every prediction the LLM makes gets tested against physical reality, including snap-fit tolerances, motor backlash, and wheel slip on the demo surface. Simulation cannot soften the errors, so the gap residuals reflect what actually happened on the floor.

The loop closes fully. The full path runs from motor telemetry to LLM reasoning to physical redesign to human build to new hardware and back to testing. Most capstone projects touch two or three of these stages; this one completes the whole circuit.

The system is multi-modal. It combines per-actuator telemetry (a 20 ms motor loop), computer vision (YOLO plus AprilTags at 8 to 10 FPS), and language reasoning. The LLM synthesizes all three to revise a single structured self-model.

The architecture is multi-LLM and adversarial. A generator agent authors the design, a critic panel attacks it, and the generator folds in the critique before any physical assembly gets committed. This mirrors real engineering review, with AI agents standing in for domain experts.

The research position is new. No published work sits at the intersection of language-authored self-models, adversarial pre-build critique, and reality correction through per-actuator telemetry. Lipson (self-models), Hart and Scassellati (symbolic self-models before LLMs), and RoboMorph (LLM design in simulation) each cover one or two of these vertices. This project covers all three.

Why It Is Worth Solving

Self-modeling is foundational to robot adaptability. A robot that cannot revise its own model cannot adapt to a new surface, a worn motor, or an unexpected load. Fixed models fail the moment deployment conditions differ from design conditions.

Language grounding makes self-models inspectable. An LLM-authored self-model reads cleanly to a human engineer, edits without retraining, and audits across generations. When something goes wrong, you can read exactly what the robot thought it could do and why it thought that.

The gap residual approach generalizes. The predicted, observed, and gap contract we define for a VEX motor works the same way for an industrial manipulator, a surgical robot, or a prosthetic limb. The problem formulation stays domain-general.

The adversarial critic pattern scales. Running a critic panel before physical assembly costs little, just a short Claude Code session. Catching a torque-budget error before a human builds Gen 4 saves hours. At industrial scale, the same pattern removes an entire class of expensive physical prototyping cycles.

Architecture

System Overview

OPERATOR (Claude Code – subscription)

- └ Generator: reads JSONL → revises self-model
- └ Critic panel: parallel agents attack design

| reads session_*.jsonl
| writes revised self-model + build instructions



Raspberry Pi 5 (data-collection layer)

- └ vision_loop.py – Camera + YOLO + AprilTags
- └ serial_bridge.py – V5 JSON → merged JSONL

| USB serial 115,200 baud



VEX V5 Brain (motor-control layer)

- └ 4× Smart Motors – torque/current/vel/pos API

How the Work Is Divided

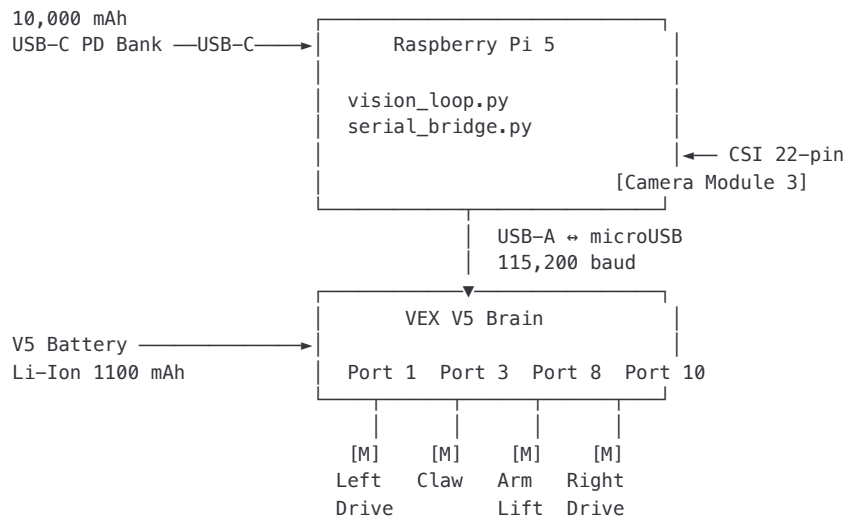
The project breaks into seven independently ownable work chunks, each with clear inputs and outputs so the team can build in parallel. The Generational Loop is shared across everyone, since it works as the integration spec rather than one person's piece. With four people and seven chunks, each person picks up more than one.

#	CHUNK	WHAT YOU OWN	INTERFACE IN	INTERFACE OUT
1	VEX V5 Brain	<code>brain_main.py</code> , motor wiring, port assignments, bumper switch config	Task commands from operator	Contract JSON lines on USB serial
2	Pi 5 Coprocessor	<code>vision_loop.py</code> , camera setup, YOLO11n NCNN, AprilTag detection	USB serial from V5; CSI from camera	<code>session_*.jsonl</code> on Pi filesystem
3	Robot Configuration	<code>parts_catalog.json</code> , typed grammar vocabulary, valid-config rules, expansion decisions	Physical kit BOM	Grammar spec that Generator and Critic both read
4	Telemetry Pipeline	JSON schema (<code>predicted</code> / <code>observed</code> / <code>gap</code> / <code>vision</code> field names), <code>serial_bridge.py</code> merge logic, JSONL format	Contract lines from V5; vision state from Pi	<code>session_*.jsonl</code> consumable by Claude
5	Self-Model & Generator	Self-model JSON schema (4 layers plus <code>reasoning</code>), Claude Code prompts and workflow for authoring and revision	<code>parts_catalog.json</code> + <code>session_*.jsonl</code>	Revised self-model JSON + BOM
6	Critic LLM Panel	Critic prompts (one per dimension), parallel invocation workflow, pass/flag output format	Proposed self-model from Generator	Structured critique for operator review
7	Aesthetic Vocabulary	Non-functional grammar (<code>body_panel</code> / <code>markings</code> / <code>appendages</code> / <code>lighting</code>), materials guide, NeoPixel SPI wiring for Pi 5	Approved self-model (per-generation identity)	Aesthetic spec appended to self-model

Dependencies worth noting

- Chunks 1 and 2 have to agree on the USB serial JSON format, so both coordinate with the Chunk 4 owner first.
- Chunks 5 and 6 both read `parts_catalog.json`, so Chunk 3 needs to stabilize before either goes far.
- Chunk 7 has no runtime dependency on chunks 1 through 6, so it can be built and tested on its own against any self-model JSON.

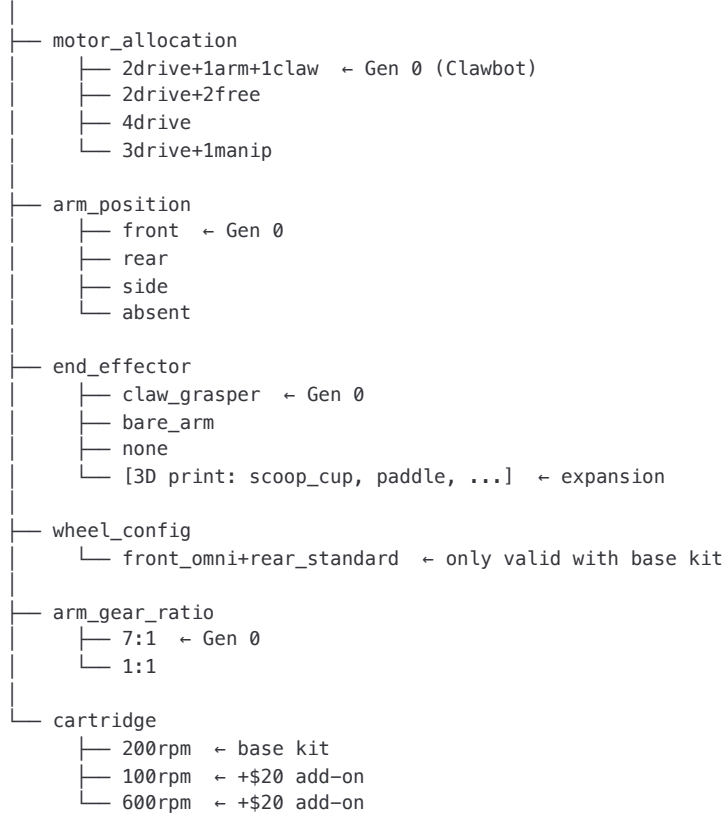
1. Hardware Platform



Two independent power supplies keep the system robust. The V5 battery powers the motors only, and the USB-C bank powers the Pi and camera. A failure of either supply leaves the other running.

2. Robot Configuration

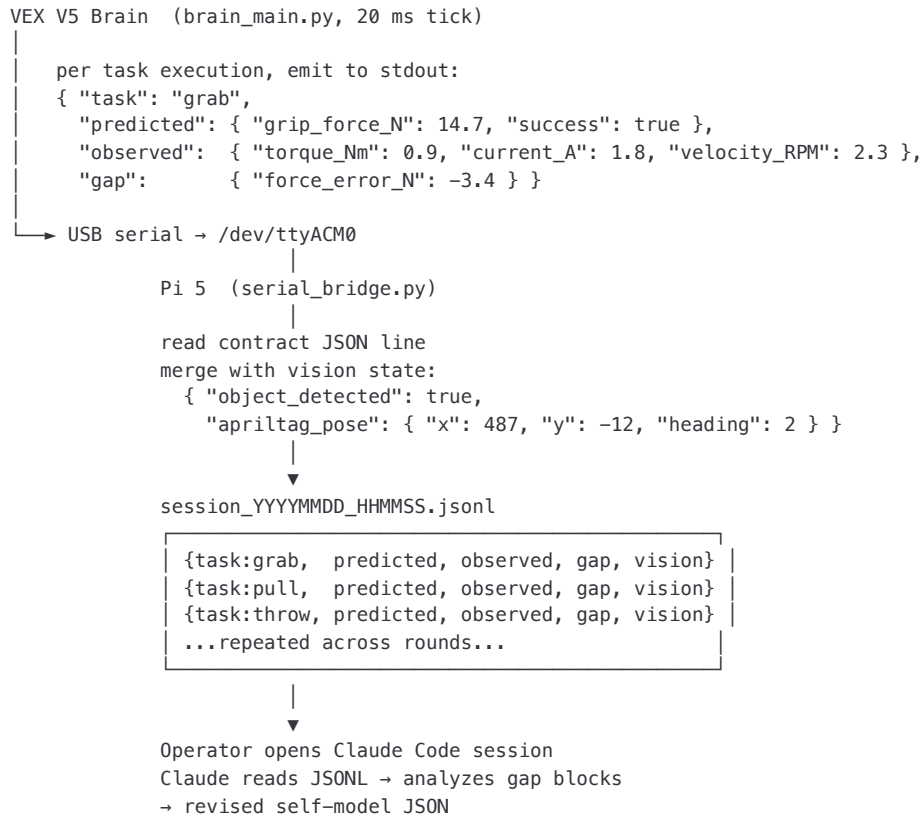
parts_catalog.json — the LLM's design vocabulary



Gen 0 telemetry gaps → LLM proposes mutation → Gen 1 config
(different from Gen 0)

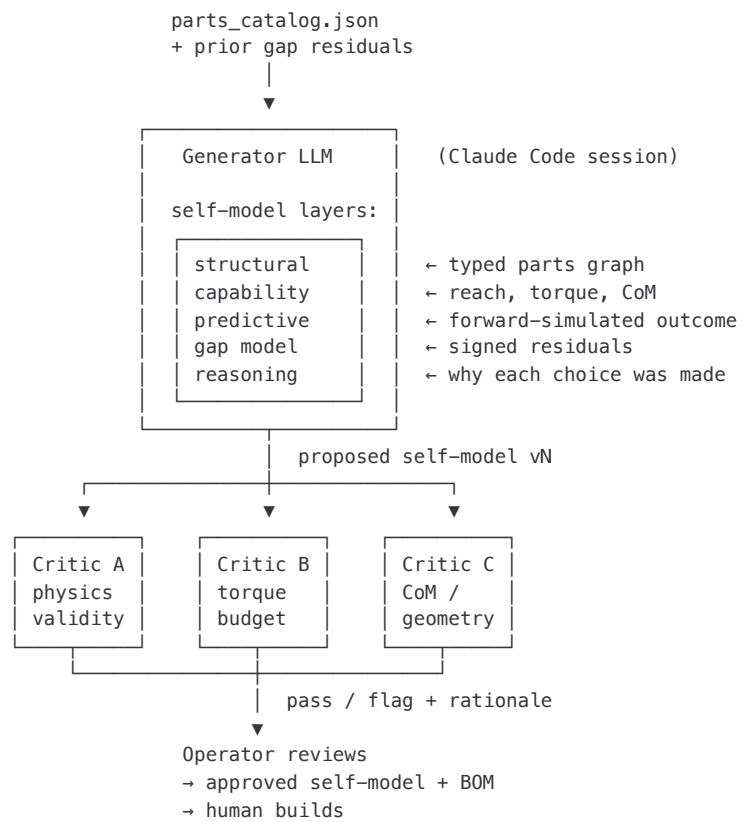
The base kit yields about 10 to 15 valid configurations. That count is small enough to work through in 3 to 5 generations and large enough to produce behavioral differences worth measuring.

3. Telemetry Pipeline



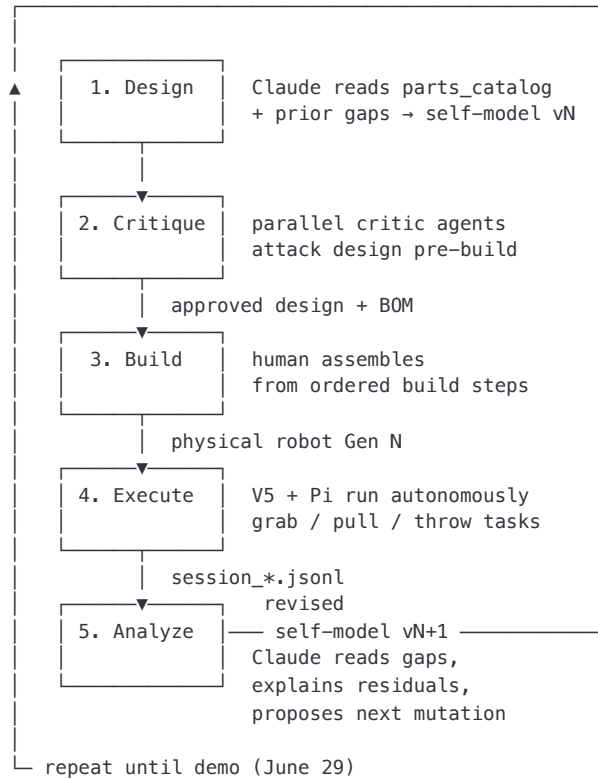
The `gap` block is the only signal Claude needs to revise the model. A platform without per-actuator feedback cannot populate `observed`, and so it cannot close the loop.

4. LLM Integration



Critic agents stay stateless. Each one reads only the proposed self-model, never the telemetry, which means we can test them against synthetic self-models before any hardware exists.

5. Generational Loop



Stage 4 is the only fully autonomous stage, where the robot runs without human intervention. Every other stage has the operator working through Claude Code.

6. Aesthetic Vocabulary

Robot = Functional Core + Aesthetic Layer

Functional (motor commands + telemetry – unchanged):

[drive_L]	[drive_R]	[arm_lift]	[claw]
-----------	-----------	------------	--------

Aesthetic (Claude-authored, unique per generation):

body_panel	—— material + position	← foam / coroplast / 3D print
surface_marks	—— tape pattern + color	← stripes / chevron / solid
appendages	—— type + position	← antennae / fins / whiskers
accent_light	—— NeoPixel pattern	← breathing / chase / pulse

Hypotheses encoded visually:

- "wide side panels" → testing mass distribution
- "forward antennae" → prioritizing forward sensing
- "Gen 3 deep blue" → hypothesis about surface friction

Attach via existing VEX 0.5" square holes – velcro, zip ties, screws.
No drilling. No modification to metal. \$0–25 per generation.
Generations become visually distinct → photographable narrative arc.

The aesthetic layer gives the generator a way to make each generation visually distinct and to encode a hypothesis in a form a camera can capture. None of it touches motor commands or telemetry contracts.

7. Key Decisions

DECISION		CHOSEN		REJECTED
Coprocessor	→	Raspberry Pi 5 3× CPU, USB-C power	x	Jetson Nano (EOL) x Jetson Orin (\$430+, 7–20V only)
Serial link	→	USB 115,200 baud no extra hardware	–	RS-485 Smart Port = Stage 2 upgrade path, not replacement
Storage	→	JSONL append-only, fast Claude reads directly	x	SQLite (slow SD writes) x CSV (no nesting)
Assembly	→	Human-in-the-loop human = formal manufacturing station	x	Full autonomy (low feasibility at capstone scale)
Localization	→	AprilTags fiducial workspace GPS, no IMU drift	x	Pure odometry (wheel slip) x Intel RealSense (cost, weight)
Design space	→	Starter Kit only ~10–15 configs exhaustible in 3–5 gens	–	Booster Kit = add-on demand (buy when loop reaches it)
LLM runtime	→	Claude Code subscription reads files directly	x	Scripted API (key mgmt, per-call billing, latency engineering)

